

Akashic Agent Query Flow

这份笔记按逻辑阶段解释：用户输入一句 query 后，Akashic Agent 从收到消息到发出回复，中间到底做了什么。

它不是完整调用栈，而是帮助理解系统设计的地图。每一阶段会标出最关键的代码位置，方便继续深挖。

0. 总览

一条普通消息大致会走这条链：

```
Channel
-> MessageBus
-> AgentLoop
-> AgentCoreRunner
-> PassiveTurnPipeline
-> BeforeTurn
-> BeforeReasoning
-> Reasoner.run_turn
-> Reasoner.run
-> AfterReasoning
-> AfterTurn
-> Outbound Bus
-> Channel.send
```

换成人话就是：

```
收到消息
-> 放进队列
-> agent loop 取出来
-> 准备会话和上下文
-> 检索记忆
-> 拼 prompt
-> 调 LLM
-> 如有工具调用就执行工具并继续推理
-> 得到最终回复
-> 写回 session / observe / memory
-> 发回用户
```

1. Channel：把外部消息变成内部消息

第一阶段是输入适配。

QQ、Telegram、CLI、IPC 这些入口各自收到消息后，不直接调用模型，而是统一转成内部的 `InboundMessage`，再丢给 `MessageBus`。

以 QQ 私聊为例：

```
QQ event
-> QQChannel._handle_private
-> MessageBus.publish_inbound(InboundMessage(...))
```

关键文件：

- `infra/channels/qq_channel.py`
- `infra/channels/telegram_channel.py`
- `infra/channels/cli.py`
- `infra/channels/ipc_server.py`
- `bus/queue.py`

这一层主要负责：

- 把不同平台的消息格式抹平
- 确定 `channel`
- 确定 `chat_id`
- 下载或整理图片等 `media`
- 生成 `session key` 所需的信息

它不做深度理解，也不决定怎么回答。

2. MessageBus：消息队列

`MessageBus` 是 `channel` 和 `agent` 之间的异步队列。

它有两个方向：

```
inbound: channel -> agent
outbound: agent -> channel
```

关键文件：

- `bus/queue.py`

核心方法：

```
publish_inbound
consume_inbound
publish_outbound
dispatch_outbound
```

逻辑上可以把它理解成：

Channel 不直接操心 agent 正在忙不忙。
Channel 只负责把消息放进 inbound queue。
AgentLoop 自己慢慢消费。

回复也是类似：

Agent 不直接操作 QQ / Telegram。
Agent 把 OutboundMessage 放进 outbound queue。
MessageBus 再分发给对应 channel 的 send 方法。

3. AgentLoop：取消息、标记 busy、交给核心处理

AgentLoop 是主循环。

它做的事情很像一个调度员：

```
while running:
    从 inbound queue 取一条消息
    建立 turn state
    标记该 session busy
    调 AgentCoreRunner.process
    结束后释放 busy
```

关键文件：

- agent/looping/core.py

重点函数：

- AgentLoop.run
- AgentLoop._process

注意：AgentLoop 本身不负责 prompt、memory、tool loop 的细节。它只是把消息交给下一层。

真正业务入口是：

```
AgentCoreRunner.process
```

4. AgentCoreRunner：区分消息类型

AgentCoreRunner 是一个很薄的分发层。

它会判断 inbound item 是哪一类：

- 普通用户消息：走 AgentCore.process

- spawn completion 等内部事件：走对应内部处理逻辑

关键文件：

- `agent/core/runner.py`

普通聊天消息会走：

```
InboundMessage  
-> AgentCore.process
```

这一层的意义是：把“普通聊天”和“内部工作项”放在同一个入口里，但保持处理逻辑分开。

5. PassiveTurnPipeline：一轮被动回复的阶段编排

普通用户 query 会进入 `PassiveTurnPipeline`。

这是理解 Akashic 被动对话最重要的一层。

关键文件：

- `agent/core/passive_turn.py`

它把一轮回复拆成几个阶段：

```
BeforeTurn  
-> BeforeReasoning  
-> Reasoning  
-> AfterReasoning  
-> AfterTurn
```

BeforeTurn

这一阶段主要准备 turn 的基础状态。

典型工作：

- 找到或创建 session
- 读取 session metadata
- 插件有机会提前拦截
- 某些命令或特殊状态可以在这里中断后续推理

逻辑上它回答的问题是：

```
这一轮对话能不能开始？  
要用哪个 session？  
有没有必要直接返回控制类消息？
```

BeforeReasoning

这一阶段负责推理前准备。

典型工作：

- 准备工具上下文
- 做 memory retrieval
- 生成 `retrieved_memory_block`
- 收集 `extra_hints`
- 插件可以追加提示或中止推理

逻辑上它回答的问题是：

为了回答这句话，模型需要哪些额外上下文？
要不要查长期记忆？
有没有额外提示要塞给本轮？

这一步产出的 `retrieved_memory_block` 会在后面拼 prompt 时进入上下文。

6. Memory Retrieval：检索相关记忆

memory 检索不是 `ContextBuilder` 自己临时想起来做的，而是在推理前的 retrieval pipeline 里完成。

关键文件：

- `agent/retrieval/default_pipeline.py`
- `plugins/default_memory/engine.py`
- `memory2/retriever.py`
- `memory2/query_rewriter.py`
- `memory2/query_builder.py`
- `memory2/hyde_enhancer.py`
- `memory2/sufficiency_checker.py`

外层入口很薄：

```
DefaultMemoryRetrievalPipeline.retrieve  
-> memory.engine.retrieve(...)
```

真正的记忆策略更多在 memory engine 和 `memory2` 模块里。

逻辑上这一步可能包含：

- 判断这句话需要哪类记忆
- 改写 query
- 构造 procedure 类查询
- 做向量检索和关键词检索
- 合并排序

- 判断检索结果够不够
- 生成注入 prompt 的文本块

它最终给主链路的不是原始数据库记录，而是一段已经整理好的：

```
retrieved_memory_block
```

后面模型会在 prompt 里看到这段内容。

7. Reasoner.run_turn：准备 history、render prompt、启动推理

进入 `DefaultReasoner.run_turn` 后，系统开始真正准备 LLM 输入。

关键文件：

- `agent/core/passive_turn.py`
- `agent/context.py`
- `agent/prompting/assembler.py`

主要步骤：

```
取 session history
-> 根据 retry plan 裁剪 history window
-> 构造 turn_injection_prompt
-> ContextBuilder.render
-> PromptAssembler.assemble
-> 得到 initial_messages
-> 调 DefaultReasoner.run
```

这里有一个很关键的点：

```
run_turn 负责“组织这一轮给模型看的输入”
run 负责“拿这个输入跑 LLM + tool loop”
```

这两个层次不要混在一起。

8. Prompt 拼装：system、history、context frame、当前消息

prompt 拼装主要由 `ContextBuilder` 和 `PromptAssembler` 完成。

关键文件：

- `agent/context.py`
- `agent/prompting/assembler.py`
- `agent/core/prompt_block.py`

`ContextBuilder` 默认会准备这些 prompt block：

- Identity
- Behavior Rules
- Memory
- Long Term Memory
- Self Model
- Recent Context
- Session Context
- Active Skills
- Skills Catalog

`PromptAssembler` 再把它分成：

```
system_sections
context_frame_sections
```

最终 message 结构大致是：

```
system prompt
history messages
context frame
current user message
```

其中：

- `system prompt` 是规则、身份、技能目录等
- `history` 是当前 session 的历史消息窗口
- `context frame` 是本轮额外上下文，比如 retrieved memory、turn injection
- `current user message` 是用户这次输入

这也是为什么历史过长或 tool result 过长时，LLM 请求体会变得很大。

9. Reasoner.run：LLM 与工具循环

`DefaultReasoner.run` 是真正的 ReAct/tool loop。

关键文件：

- `agent/core/passive_turn.py`
- `agent/tools/registry.py`
- `agent/tool_runtime.py`
- `agent/tools/tool_search.py`

逻辑大概是：

```
messages = initial_messages
tool_chain = []
tools_used = []
```

```
for iteration in max_iterations:
    BeforeStep
    LLM.chat(messages, visible_tools)

    if response has no tool_calls:
        return final reply

    append assistant tool_calls to messages
    execute each tool_call
    append tool result to messages
    record tool_chain group
    AfterStep
    append loop_state hint
```

也就是说，只要模型还在调用工具，系统就会继续：

```
LLM -> tool -> LLM -> tool -> LLM
```

直到模型给出最终文本回复，或者达到最大迭代次数。

Tool Search

工具不是一定全量暴露给模型。

当 `tool_search` 开启时，模型先看到一批 `always-on` 工具。如果它需要别的工具，可能要先调用：

```
tool_search(query="select:xxx")
```

然后下一轮才会解锁更多工具 schema。

这相当于工具层的能力路由。

Tool Chain

工具执行后会记录 `tool_chain`。

一组 tool chain 通常包含：

- 本轮 LLM 输出的文字
- 本轮调用了哪些工具
- 每个工具的参数
- hook trace
- result preview
- reasoning content

它主要用于 dashboard 回放和调试。

10. 得到最终回复

当某一轮 LLM 没有再返回 tool calls，而是返回文本时，系统认为本轮推理完成。

这时会构造：

```
ReasonerResult
```

里面包括：

- `reply`
- `thinking`
- `metadata.tools_used`
- `metadata.tool_chain`
- `metadata.visible_names`
- `metadata.react_stats`

关键文件：

- `agent/core/passive_turn.py`

这里的 `react_stats` 会记录本轮输入 token 估算，比如：

- `iteration_count`
- `turn_input_sum_tokens`
- `turn_input_peak_tokens`
- `final_call_input_tokens`

11. AfterReasoning：回复后、提交前处理

推理结束后会进入 `AfterReasoning`。

这一阶段会把推理结果转成 outbound 结果，并给插件/模块一个处理机会。

逻辑上它回答：

```
模型已经给出结果了。  
在真正写回和发送前，还要不要改一下？
```

可能发生的事情包括：

- 构造 outbound message
- 附加 metadata
- 插件后处理
- 某些安全或格式处理

关键文件：

- `agent/core/passive_turn.py`
- `agent/lifecycle/phases/after_reasoning.py`

12. AfterTurn：提交 session、触发副作用、派发回复

AfterTurn 是一轮对话真正落地的地方。

典型工作：

- 写入 user message
- 写入 assistant message
- 保存 tool_chain
- 更新 session metadata
- 触发 observe
- 触发 memory 后处理
- dispatch outbound

关键文件：

- `agent/lifecycle/phases/after_turn.py`
- `session/manager.py`
- `session/store.py`
- `plugins/00_observe/writer.py`
- `plugins/default_memory/engine.py`

这里要注意：

`session` 写回不是 `AgentLoop` 直接做的。
它发生在 `PassiveTurnPipeline` 的 `AfterTurn` 阶段。

`session` 存储层会把每条 message 写入 SQLite，其中包括：

- role
- content
- tool_chain
- extra
- timestamp

13. Outbound：把回复送回 channel

最终回复不是由 `reasoner` 直接调用 QQ API。

它会走 outbound bus：

```
AfterTurn
-> OutboundPort.dispatch
-> MessageBus.publish_outbound
```

```
-> MessageBus.dispatch_outbound  
-> channel.send
```

关键文件：

- `bus/queue.py`
- `agent/turns/outbound.py`
- `infra/channels/qq_channel.py`
- `infra/channels/telegram_channel.py`

以 QQ 为例，最后会进入：

```
QQChannel.send  
-> api.send_private_text / api.send_group_text
```

14. Observe 与 Memory 是旁路，但很重要

在主回复链之外，还有两个重要旁路。

Observe

Observe 更像调试和回放系统。

它记录：

- turn started
- tool call started
- tool call completed
- reply
- rag query
- 其他事件

关键文件：

- `plugins/00_observe`
- `bus/events.py`
- `bus/event_bus.py`

Memory

Memory 不是每次都同步阻塞主回复全部完成。

它会在 turn 后做 consolidation、抽取长期记忆、写 profile/procedure/preference 等。

关键文件：

- `plugins/default_memory/engine.py`
- `memory2/memorizer.py`
- `memory2/post_response_worker.py`

- `memory2/store.py`

逻辑上：

主链路负责回答当前问题。
`memory` 旁路负责把有价值的内容沉淀下来，供未来检索。

15. 从“学习逻辑”的角度怎么记

可以把 Akashic 一轮 query 分成七个大阶段：

第一阶段：输入标准化

外部平台消息变成统一的 `InboundMessage`。

第二阶段：调度

`MessageBus` 排队，`AgentLoop` 消费并标记 busy。

第三阶段：turn 准备

`PassiveTurnPipeline` 建立 session、跑 BeforeTurn、BeforeReasoning。

第四阶段：上下文准备

检索 memory，拼 system prompt、history、context frame、current message。

第五阶段：推理执行

LLM 决定回答还是调工具。若调工具，执行工具并继续下一轮。

第六阶段：结果提交

构造最终回复，写 session，记录 tool chain，触发 observe 和 memory 后处理。

第七阶段：发送

通过 outbound bus 回到对应 channel，最终发给用户。

16. 最值得先看的文件顺序

如果你想按学习路径读代码，建议顺序是：

1. `bus/queue.py`
2. `infra/channels/qq_channel.py`
3. `agent/looping/core.py`
4. `agent/core/runner.py`
5. `agent/core/passive_turn.py`
6. `agent/context.py`
7. `agent/prompting/assembler.py`

8. `agent/retrieval/default_pipeline.py`
9. `plugins/default_memory/engine.py`
10. `session/manager.py`
11. `agent/lifecycle/phases/after_turn.py`

其中最核心的是：

```
agent/core/passive_turn.py
```

它是被动对话主链路的中轴。